

iPOS Restaurant System

Dokumentácia k semestrálnemu projektu na predmet

Vývoj aplikácií s viacvrstvovou architektúrou

Maximilián Dermek

Richard Gajarský

Zsófia Gergely

Filip Hromada

Bálint Janik

Dominik Jurkas

Lucia Kadlecová

Matej Kutka

Sabina Matejčíková

Samuel Tvrdoň

Michal Weis

LS 2025/26

Obsah

1. Úvod	3
1.1 Popis systému	3
1.2 Cieľoví používatelia	3
2. Manažment projektu	3
3. Použité technológie	4
4. Architektúra aplikácie	4
4.1 MVC architekturný vzor	4
4.2 Návod na spustenie	5
Kroky spustenia	5
4.2.1 Spustenie z JAR súboru	6
Predpoklady	6
4.2.2 Spustenie zo zdroja	6
Predpoklady	6
4.3 Diagramy architektúry	8
4.3.1 Diagram komponentov	8
4.3.2 Vrstvové hľadisko	9
4.3.3 Technické hľadisko	10
4.3.4 Hľadisko použitia aplikácie	11
5. Hlavné funkcie	12
5.1 Riadenie objednávok	12
5.2 Inventár a zásoby	12
5.3 Finančné riadenie	12
5.4 Personálny manažment	12
5.5 Kuchynský pohľad	13
6. Databázová štruktúra	13
7. Testovacie účty	14
8. Splnenie definovaných požiadaviek	14
Kolekcie (Collections)	14
Logovanie (Logging)	14
Lokalizácia EN-SK	15
XML Import a export	15
Regulárne výrazy (Regex)	16
JDBC	16
Ošetrovanie/validácia vstupov + bezpečnosť	17
GUI Aplikácia (JavaFX)	17
User roles	17
5-8 obrazoviek (Screen Count)	18
Lombok	18
9. Záver	19

1. Úvod

1.1 Popis systému

iPOS Restaurant System je aplikácia pokladničného a prevádzkového systému pre reštaurácie, kaviarne, bary. Aplikácia rieši každodennú prácu obsluhy, kuchyne a manažmentu podniku. Umožňuje pracovať so stolmi, prijímať objednávky, evidovať položky v menu a mnohé ďalšie funkcie.

Celé zadanie projektu: https://github.com/miroslav-reiter/VAVA_JAVA

1.2 Cieľoví používatelia

Admin - Má prístup k manažérskym funkciám, správe používateľov, inventáru, menu, financiám a rozloženiu stolov.

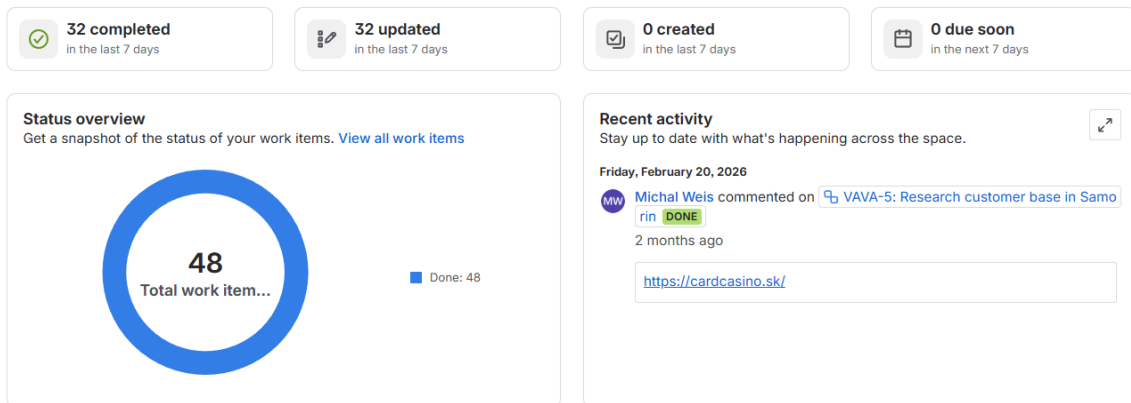
Čašník - Pracuje so stolmi, vytvára objednávky, pridáva položky, spracúva platby.

Kuchár - Vidí objednávky určené do kuchyne a označuje jednotlivé položky ako dokončené.

2. Manažment projektu

Na vývoj aplikácie sme používali nástroj GitHub, kde sme do spoločného repozitáru spoločne pridávali naše časti kódu. Každá zmena bola najskôr implementovaná na separátnom branch označeným číslom tasku v Jire, a až po schválení bola spojená do Main vetvy, ktorá predstavovala aktuálnu, stabilnú a spustiteľnú verziu projektu. Repozitár je verejne dostupný: <https://github.com/Balrasko/VAVA>.

Na rozdelenie úloh v projekte sme používali softvér Jira. Pracovali sme v 4 šprintoch, počas ktorých sme splnili 170 taskov. Každý piatok sme mali priebežné meetingy, kde sme si zdieľali spoločný progress a definovali ďalšie úlohy na budúci týždeň. Pracovali sme spoločne a pravidelne, čím sme presiahli naše prvotné očakávania funkcionality aplikácie.



3. Použité technológie

- **Programovací jazyk:** Java 25
- **Build nástroj:** Apache Maven 3.8+
- **UI Framework:** JavaFX 25.0.2
- **Databázový Driver:** PostgreSQL 42.7.3
- **Database Management System:** PostgreSQL 17
- **Dependency Injection:** Google Guice 7.0.0
- **Kód generovanie:** Project Lombok 1.18.44
- **Kontajnerizácia:** Docker Compose

4. Architektúra aplikácie

4.1 MVC architekturný vzor

Aplikácia je postavená na klasickom **Model-View-Controller (MVC)** architektonickom vzore rozšírenom o ďalšie vrstvy:

- **Model:** Doménové objekty, DTO a entity reprezentujúce obchodnú logiku
- **View:** FXML súbory definujúce UI rozloženie a CSS štýly pre vizuálny dizajn

- **Controller**: JavaFX controllers spracúvajúce používateľské interakcie
- **Service**: Business logika, validačné pravidlá a orchestrácia operácií
- **DAO (Data Access Object)**: Abstrakcia databázových operácií
- **DTO (Data Transfer Object)**: Objekty pre prenos dát medzi vrstvami

4.2 Návod na spustenie

Zdrojový kód ako aj spustiteľný **.jar** súbor nájdete v repozitári tu: <https://github.com/Balrasko/VAVA/tree/main> alebo v odovzdanom .zip súbore

Kroky spustenia

1. Klonovanie repozitára alebo odovzdaný .zip

```
git clone https://github.com/Balrasko/VAVA.git
```

```
cd VAVA
```

2. Nastavenie premenných prostredia

Vytvorenie .env súboru na základe .env.example:

```
cp .env.example .env
```

3. Spustenie databázy

- *Pre vývojové prostredie spustíte:*

```
docker-compose up -d
```

- Keby ste chceli kontajner zastaviť a odstrániť:

```
docker-compose down -v
```

- *Pre produkčné prostredie s perzistentnými dátami spustíte:*

```
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

- Keby ste chceli zmazať produkčné dáta a kontajner:

```
docker-compose -f docker-compose.yml -f  
docker-compose.prod.yml down -v
```

4.2.1 Spustenie z JAR súboru

Predpoklady

- Java Runtime Environment (JRE) 25+
- Docker Compose

Spustenie aplikácie z existujúceho JAR súboru:

```
java -jar path-to-existing-jar-file
```

(ex. ./target/RestaurantApp.jar)

4.2.2 Spustenie zo zdroja

Predpoklady

- Git pre klonovanie repozitára
- Java Development Kit (JDK) 25+
- Apache Maven 3.8+
- Docker Compose

Kompilácia a spustenie aplikácie z koreňového adresára:

```
mvn clean javafx:run
```

Alternatíva: Zbuildovanie a spustenie aplikácie z JAR súboru z koreňového adresára:

```
mvn clean package
```

```
java -jar target/RestaurantApp.jar
```

Keď JAR beží na hostiteľskom počítači, ponechajte predvolený hostiteľ a port databázy v .env:

RESTAURANT_DB_HOST=localhost

RESTAURANT_DB_PORT=5433

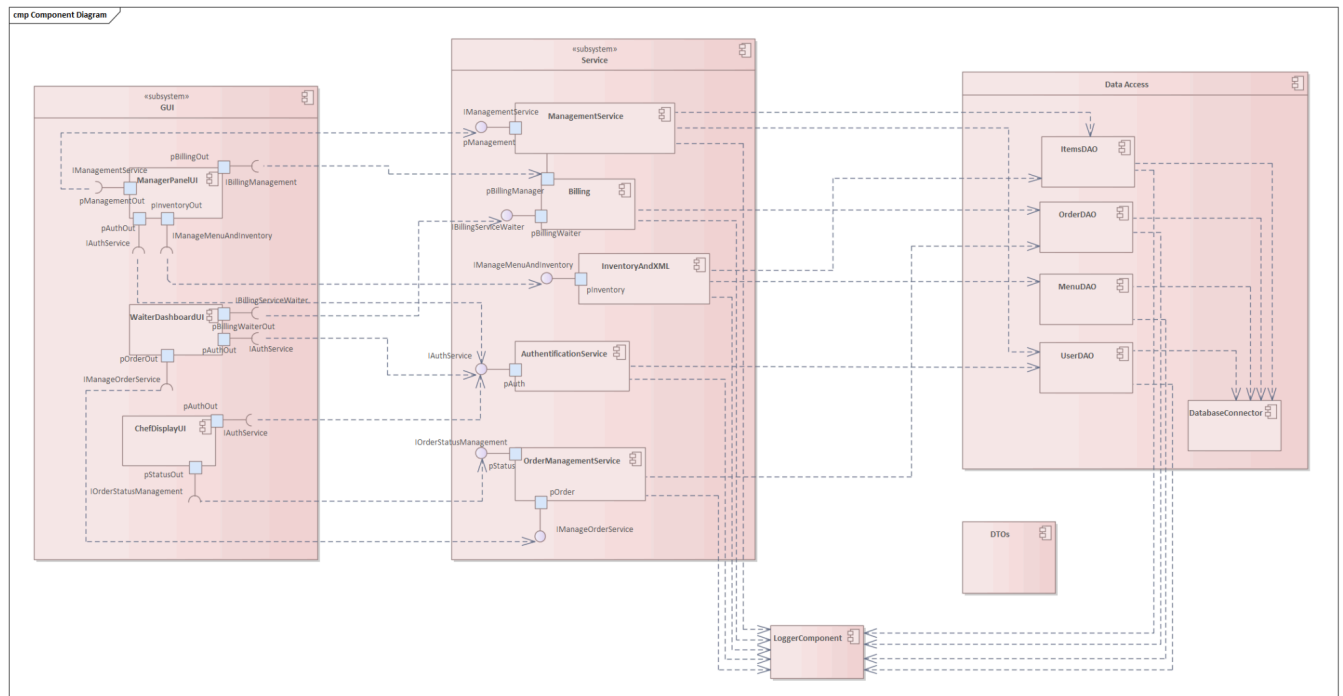
Ak bude Java aplikácia niekedy presunutá do rovnakej Docker Compose siete, použite namiesto toho:

RESTAURANT_DB_HOST=db

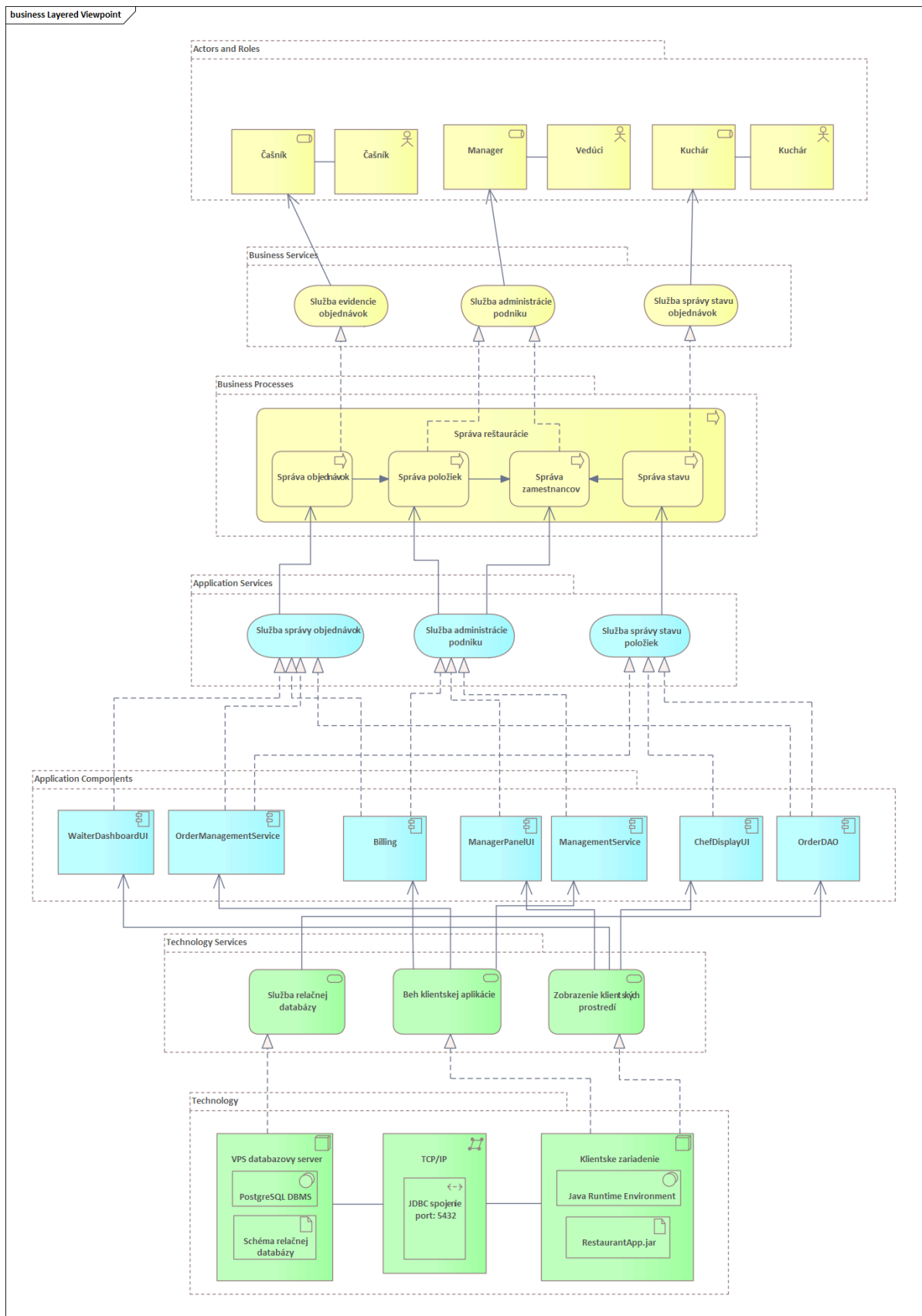
RESTAURANT_DB_PORT=5432

4.3 Diagramy architektúry

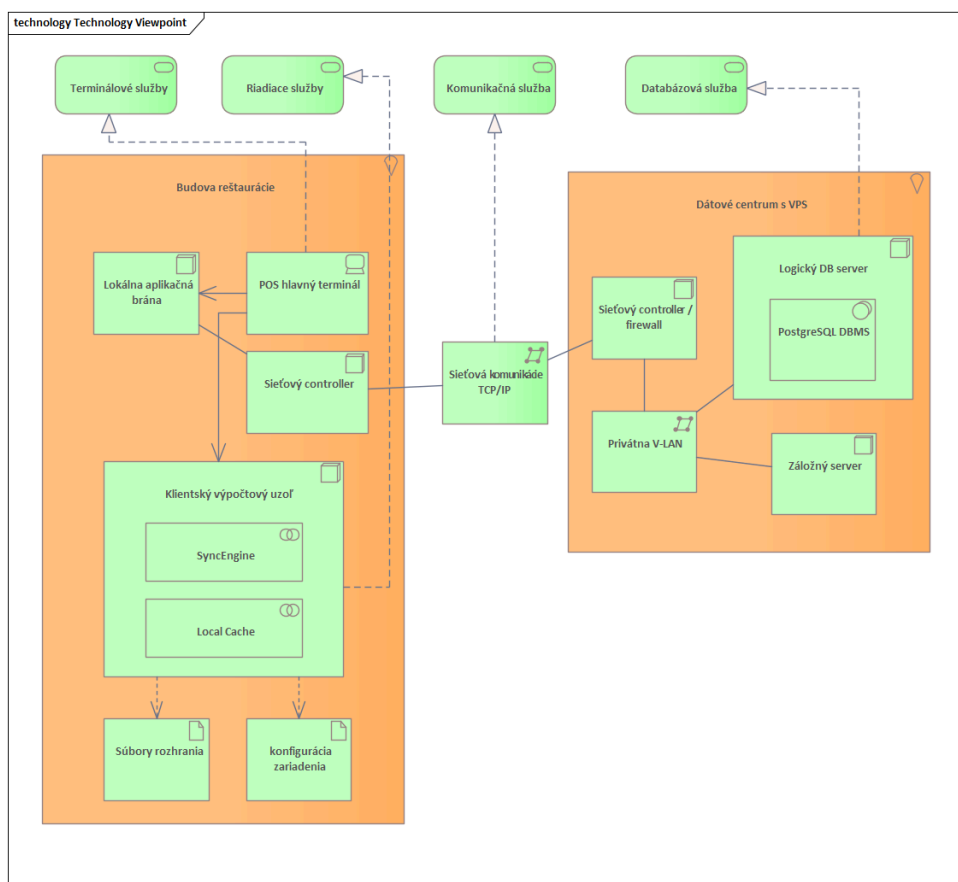
4.3.1 Diagram komponentov



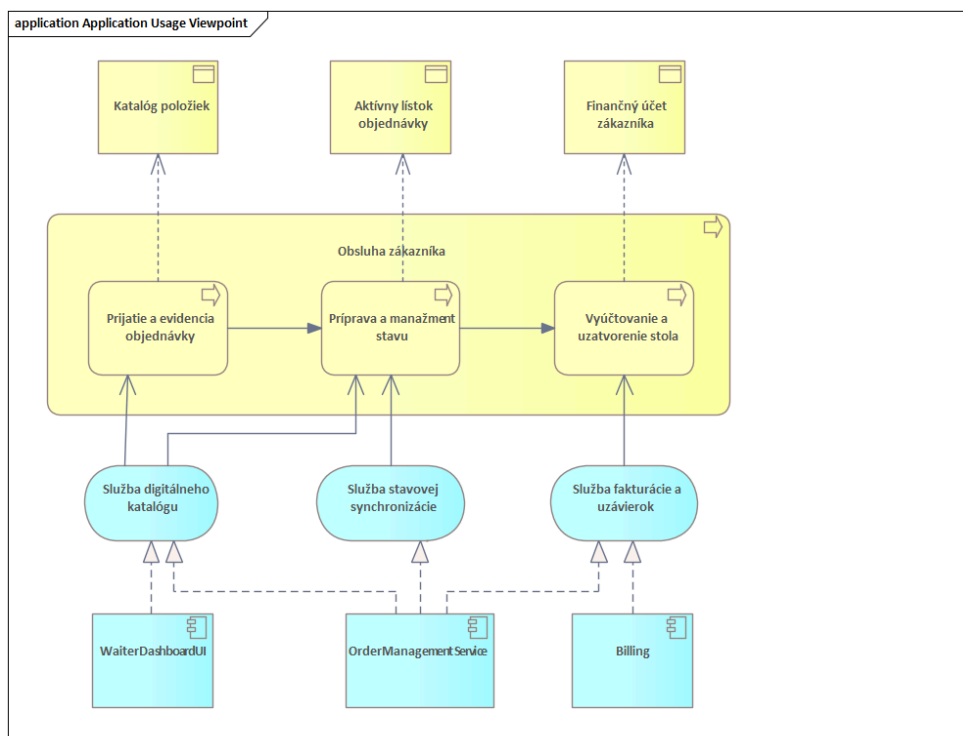
4.3.2 Vrstvové hľadisko



4.3.3 Technické hľadisko



4.3.4 Hľadisko použitia aplikácie



5. Hlavné funkcie

5.1 Riadenie objednávok

- **Vytváranie objednávok:** Intuitívne rozhranie pre výber položiek z menu
- **Stavový manažment:** Automatické sledovanie stavu (Prijatá → V príprave → Hotová → Podávaná)
- **Priradenie ku stolom:** Vizualizácia rozloženia reštaurácie s interaktívnou mapou stolov podporujúca zónovanie
- **Modifikácie položiek:** Poznámky k jedlám
- **Rozdelenie účtu:** Možnosť rozdelenia jednej objednávky na viac platieb

5.2 Inventár a zásoby

- **Sledovanie surovín:** Reálne množstvá všetkých ingrediencií
- **Filtrovanie produktov:** Vyhľadávanie podľa názvu a filtrovanie podľa množstva, zoradenie podľa ID, názvu, množstva

5.3 Finančné riadenie

- **Pokladňa:** Správa hotovosti, vklady a výbery
- **Platobné metódy:** Hotovosť, karta
- **Finančné reporty:** Denné prehľady tržieb cez uzávierku, a prehľady za vybrané časové obdobie v manažer rozhraní

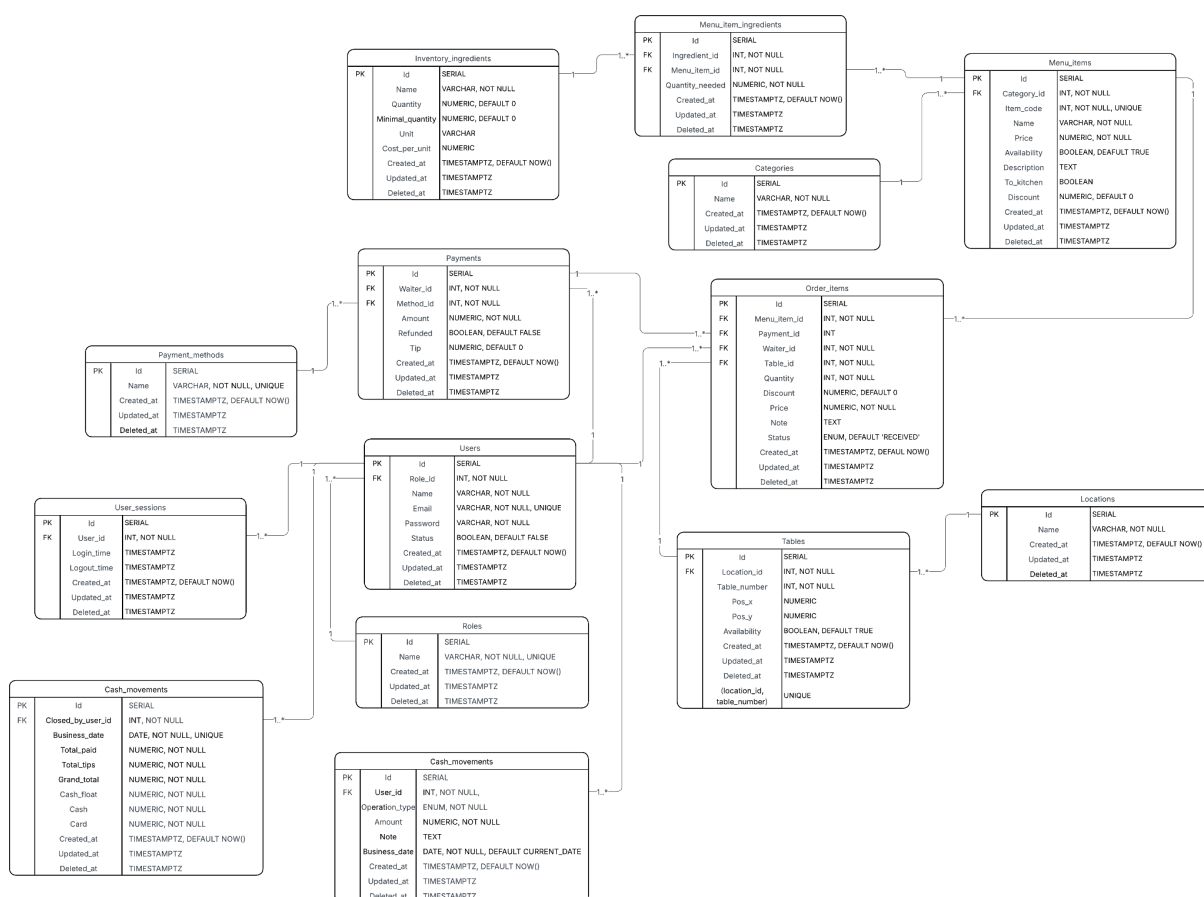
5.4 Personálny manažment

- **Správa používateľov:** Registrácia, úprava a deaktivácia účtov
- **Role a oprávnenia:** Manažér (plný prístup), Čašník (obmedzený prístup), Kuchár (prehľad objednávok)
- **Autentifikácia:** Bezpečné prihlásenie s hashovanými heslami

5.5 Kuchynský pohľad

- **Aktívne objednávky:** Zobrazenie všetkých rozpracovaných objednávok
- **Aktualizácia stavu:** Rýchle prepínanie medzi stavmi prípravy
- **Komunikácia:** Poznámky medzi čašníkmi a kuchármi

6. Databázová štruktúra



7. Testovacie účty

Rola	Email	Heslo	Popis
Admin	admin@vava.com	admin123	Plný prístup ku všetkým funkciám mimo chef view
Chef	chef1@vava.com	chef123	Prístup ku chef view
Čašník	waiter1@vava.com	waiter123	Prijímanie objednávk, správa stolov

8. Splnenie definovaných požiadaviek

Kolekcie (Collections)

V aplikácií sme používali vhodné dátové typy z Java Collections Framework. Napríklad ArrayList v KitchenService. Tento zoznam obsahuje všetky položky jedného stola, ktoré sú určené na prípravu pre kuchyňu.

```
for (OrderItem item : orderItemDao.getUnpaidOrderItems()) {  
    if (!isKitchenItem(item, menuItemsById)) {  
        continue;  
    }  
    if (!KITCHEN_BOARD_STATUSES.contains(item.getStatus())) {  
        continue;  
    }  
    itemsByTable.computeIfAbsent(item.getTableId(), key -> new ArrayList<>()).add(item);  
}
```

Logovanie (Logging)

Na loggovanie sme ako API použili knižnicu *SLF4J* a ako backend sme využili knižnicu *logback*. Logovanie je implementované ako výpis do terminálu cez console appender aj do súboru cez file appender. Všetky dôležité akcie sú zaznamenávané ako úroveň *INFO* a loggy z externých knižníc sú zaznamenané ako *WARN*.

```

<appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>logs/vava.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>logs/vava.%d{yyyy-MM-dd}.log</fileNamePattern>
    <maxHistory>30</maxHistory>
  </rollingPolicy>
  <encoder>
    <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
  </encoder>
</appender>

```

Lokalizácia EN-SK

Implementovali sme podporu pre anglický (ENG) a slovenský (SK) jazyk pomocou ResourceBundle. Vytvorili sme súbory messages_en.properties. Na správu lokalizácie používame utilitnú triedu **I18n**, ktorá načítava správny jazykový bundle, prepína jazyk cez toggleLocale() a poskytuje metódu I18n.t("kľúč") na získanie preloženého textu.

```

public final class I18n {
  private static final String BUNDLE_NAME = "i18n.messages";
  private static final Set<String> SUPPORTED_LANGUAGES = Set.of("en", "sk");

```

XML Import a export

Import a export XML súborov je implementovaný pre obrazovky Finances a Inventory v manažer panely. Táto ukážka kódu je konkrétne export z Inventory, kde vytvoríme XML súbor obsahujúci všetky suroviny ktoré sú nahrané v databáze. Pre túto obrazovku vieme aj načítať XML súbor, ktorý uloží suroviny do databázy pokiaľ má vstupný súbor správny formát.

```

@Override
public void exportToXml(Path path, List<InventoryIngredient> items) throws IOException {
    DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
    try {
        factory.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);

        DocumentBuilder builder = factory.newDocumentBuilder();
        Document document = builder.newDocument();

        Element rootElement = document.createElement("inventory");
        document.appendChild(rootElement);

        for (InventoryIngredient item : items) {
            Element itemElement = document.createElement("item");
            rootElement.appendChild(itemElement);

            appendText(document, itemElement, "id", String.valueOf(item.getId()));
            appendText(document, itemElement, "name", safeValue(item.getName()));
            appendText(document, itemElement, "quantity", formatDecimal(item.getQuantity()));
            appendText(document, itemElement, "minimal_quantity", formatDecimal(item.getMinimalQuantity()));
            appendText(document, itemElement, "unit", safeValue(item.getUnit()));
            appendText(document, itemElement, "cost_per_unit", formatDecimal(item.getCostPerUnit()));
            appendText(document, itemElement, "status", getStatus(item).name());
        }

        TransformerFactory transformerFactory = TransformerFactory.newInstance();
        Transformer transformer = transformerFactory.newTransformer();
        transformer.setOutputProperty(OutputKeys.INDENT, "yes");
        transformer.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
        transformer.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");

        transformer.transform(new DOMSource(document), new StreamResult(path.toFile()));
    } catch (ParserConfigurationException | TransformerException e) {
        throw new IOException("Failed to write inventory XML", e);
    }
}

```

Regulárne výrazy (Regex)

Regex v aplikácií využívame Regex na vyhľadavanie či validáciu emailu. Email verifikujeme podľa komplexného Regexu, ktorý je verejne dostupný na: <https://emailregex.com/>.

```

5 public final class ValidationUtils {
6
7     // https://emailregex.com/index.html
8     private static final Pattern EMAIL_PATTERN = Pattern.compile(
9         "(?:[a-z0-9!#$%&'*/+=?^_`{|}~]+(?:\\.[a-z0-9!#$%&'*/+=?^_`{|}~]+)*|\"(?:[\\x01-\\x08\\x0b\\x0c\\x0e-\\x1f\\x21\\x23-\\x5b\\x5d-\\x7f]|\\\\\\\\[\\x01-\\x09\\x0b\\x0c\\x0e-\\x7f])*\")@(?:(?:[a-z0-9](?:[a-z0-9]|(?:[\\x01-\\x08\\x0b\\x0c\\x0e-\\x7f]|\\\\\\\\[\\x01-\\x09\\x0b\\x0c\\x0e-\\x7f]))?\\.)+[a-z0-9](?:[a-z0-9]|(?:[\\x01-\\x08\\x0b\\x0c\\x0e-\\x7f]|\\\\\\\\[\\x01-\\x09\\x0b\\x0c\\x0e-\\x7f]))?|(?:(?:25[0-5]|2[0-4][0-9]|0[0-9][0-9])?(?:25[0-5]|2[0-4][0-9]|0[0-9][0-9])?(?:[a-z0-9](?:[a-z0-9]|(?:[\\x01-\\x08\\x0b\\x0c\\x0e-\\x7f]|\\\\\\\\[\\x01-\\x09\\x0b\\x0c\\x0e-\\x7f]))?)+)))(?:[a-z0-9](?:[a-z0-9]|(?:[\\x01-\\x08\\x0b\\x0c\\x0e-\\x7f]|\\\\\\\\[\\x01-\\x09\\x0b\\x0c\\x0e-\\x7f]))?)+)");
10
11     public static boolean isValidEmail(String email) {
12         return email != null && EMAIL_PATTERN.matcher(email).matches();
13     }
14 }
15
16
17

```

JDBC

V projekte používame JDBC na priamu komunikáciu s PostgreSQL databázou. Pripojenie vytvára trieda ConnectionFactory, ktorá načíta PostgreSQL driver a vytvorí Connection pomocou DriverManager.

Metóda getUnpaidOrderItems() slúži na načítanie všetkých položiek objednávok, ktoré ešte neboli zaplatené. Z databázy vyberá záznamy z tabuľky order_items, kde je payment_id IS NULL.

```

@Override
public List<OrderItem> getUnpaidOrderItems() {
    log.info("Fetching all unpaid order items");
    String sql = """
        SELECT id, menu_item_id, payment_id, waiter_id, table_id, quantity, discount, price, note,
               status, created_at, updated_at
        FROM order_items
        WHERE payment_id IS NULL
        ORDER BY created_at ASC, id ASC
        """;

    List<OrderItem> orderItems = new ArrayList<>();
}

```

Ošetrovanie/validácia vstupov + bezpečnosť

Bezpečnosť aplikácie je riešená ochranou vstupov proti SQL Injections v DAO súboroch pomocou PreparedStatements, a placeholdermi. Heslá užívateľov sú šifrované pomocou BCrypt, a ukladané v databáze.

```

BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(SecurityConfig.BCRYPT_STRENGTH);
String hash = encoder.encode(user.getPasswordHash());

```

GUI Aplikácia (JavaFX)

Celá aplikácia je desktopová GUI aplikácia postavená na technológii JavaFX. Používame FXML súbory (*.fxml) na deklaratívne definovanie štruktúry a komponentov jednotlivých obrazoviek. Každý FXML súbor má priradený Java Controller (*Controller.java), ktorý obsahuje logiku na spracovanie udalostí a interakciu s používateľom. Vizualný štýl je definovaný externe pomocou CSS (style.css).

User roles

V aplikácii používame používateľské roly na oddelenie prístupov a funkcionalít. V databáze sú definované roly ADMIN, WAITER a CHEF, pričom každému používateľovi je priradené role_id.

```

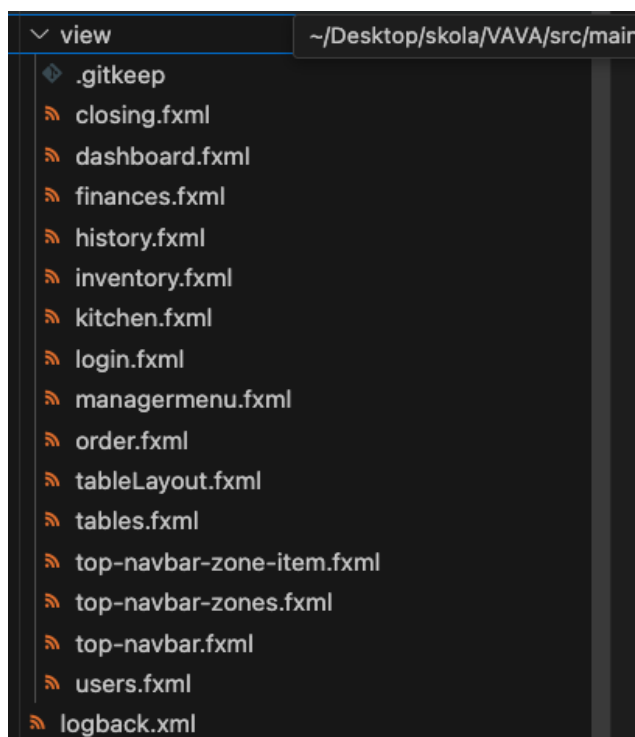
@Override
public String getRoleName(int roleId) {
    return switch (roleId) {
        case 1 -> I18n.t("role.admin");
        case 2 -> I18n.t("role.waiter");
        case 3 -> I18n.t("role.chef");
        default -> I18n.t("role.user");
    };
}

```

5-8 obrazoviek (Screen Count)

Aplikácia je rozdelená na viacero samostatných JavaFX obrazoviek definovaných vo FXML súboroch v priečinku `src/main/resources/view`. Medzi hlavné obrazovky patria napríklad `login.fxml`, `dashboard.fxml`, `tables.fxml`, `order.fxml`, `kitchen.fxml`, `inventory.fxml`, `finances.fxml` a `users.fxml`.

Prepínanie medzi obrazovkami rieši `DashboardController`.



Lombok

V projekte používame knižnicu Lombok na zjednodušenie modelových a DTO tried. Pomocou anotácií ako `@Data`, `@NoArgsConstructor` a `@AllArgsConstructor`

automaticky generujeme getters, setters, konštruktory a ďalšie bežné metódy, takže triedy ako MenuItem alebo Role zostávajú kratšie a prehľadnejšie.

```
5
9  @Data
9  @NoArgsConstructor
1  @AllArgsConstructor
2  ✓ public class MenuItem {
3      private int id;
4      private int categoryId;
5      private int itemCode;
6      private String name;
7      private BigDecimal price;
8      private boolean availability;
9      private String description;
9      private boolean toKitchen;
1     private BigDecimal discount;
2     private OffsetDateTime createdAt;
3     private OffsetDateTime updatedAt;
4     private OffsetDateTime deletedAt;
5 }
```

9. Záver

iPOS Restaurant System predstavuje funkčnú aplikáciu POS systému pre gastronomickú prevádzku. Aplikácia demonštruje prepojenie desktopového používateľského rozhrania v JavaFX s relačnou databázou PostgreSQL. Vďaka rozdeleniu na controller, service, DAO a model vrstvu je projekt prehľadný a pripravený na ďalšie rozširovanie.

Aplikácia pokrýva základné procesy reštauračnej prevádzky: prihlásenie používateľov, prácu so stolmi, tvorbu objednávok, kuchynský pohľad, platby, inventár, históriu a dennú uzávierku. Dokumentácia slúži ako stručný technický a funkčný opis riešenia postaveného na technológiách Java a DBMS.